



THE NATIONAL INSTITUTE OF ENGINEERING
An Autonomous Institute under Visvesvaraya Technological University, Belagavi
Department of Computer Science & Engineering



CERTIFICATE

This is to certify that the DBMS Minor project work entitled "Real-Time Food Operations Analytics Dashboard" is a bonafide work carried out by Prajwal Sahu, Pavan C, Niteesh Srivathsa, Prajwal Mantur, Prajwal Babu K S in partial fulfillment for the award of degree of Bachelor of Engineering in Computer Science and Engineering, of Visvesvaraya Technological University, Belagavi, during the year 2025-26. It is certified that all corrections / suggestions indicated during internal assessment have been incorporated and the corrected copy has been deposited in the department library. This minor project report has been approved in partial fulfillment for the award of the said degree as per academic regulations of The National Institute of Engineering (Autonomous Institution).

Sumana K M
Assistant Professor
Dept. of CSE, NIE, Mysuru

Dr Anitha R
Professor and Head
Dept. of CS&E, NIE, Mysuru

ABSTRACT

This project presents the design and development of a Real-Time Operations Analytics Dashboard for a food delivery platform modeled on Zomato, built as part of the Database Management Systems (DBMS) course. The system unifies fragmented operational data restaurants, delivery agents, and orders into a normalized PostgreSQL database, exposes it through a RESTful API built with Node.js, Express, and Drizzle ORM, and presents it through an interactive Next.js web dashboard powered by a MapLibre GL geospatial map.

The dashboard renders live-animated delivery agent movement along real road routes via OSRM routing, displays restaurant and order pins on a Bengaluru city map, and provides entity drill-down modals delivering a centralized operational command center for food delivery analytics. Key DBMS concepts applied include Third Normal Form (3NF) schema design, SQL JOIN queries, aggregations, indexing, enum-typed columns, and transaction management.

ACKNOWLEDGMENT

We sincerely owe our gratitude to all persons who helped and guided us in completing this minor project.

We are thankful to Dr. Negendra Parashar, Principal, NIE College, Mysuru, for all the support he has rendered.

We thank Dr. Anitha R, Professor and Head, Department of Computer Science and Engineering, for her constant support and encouragement throughout the tenure of this minor project work.

We would like to sincerely thank our guides, the Lab Lecturers, Assistant Professors, Department of Computer Science and Engineering, for providing relevant information, valuable guidance and encouragement to complete this minor project.

Prajwal Sahu : 4NI24CS150

Pavan C : 4NI24CS147

Niteesh Srivathsa : 4NI24CS143

Prajwal Mantur : 4NI24CS155

Prajwal Babu K S : 4NI24CS153

TABLE OF CONTENTS

CONTENTS	PAGE NO.
Abstract	I
Acknowledgement	II
Table of Contents	III
List of Figures	IV
Chapter 1 – Introduction	1
1.1 Background	1
1.2 Purpose	2
1.3 Objectives	2
1.4 Existing System and Disadvantages	3
1.5 Proposed System and Advantages	3
Chapter 2 – Literature Review	5
Chapter 3 – System Requirements	7
3.1 Hardware Requirements	7
3.2 Software Requirements	7
3.3 Functional Requirements	8
3.4 Non-Functional Requirements	9
Chapter 4 – System Architecture	10
4.1 System Design	10
4.2 ER Diagram / Schema Diagram	11
Chapter 5 – Implementation	12
5.1 Steps	12
5.2 DBMS Concepts Applied	14
Chapter 6 – Results and Screenshots	16
Conclusion and Future Enhancement	18
References	19

LIST OF FIGURES

FIG. NO.	DESCRIPTION	PAGE NO.
1	System Architecture – Component Relationship Diagram	10
2	Entity-Relationship (ER) Diagram	11
3	Database Schema (3NF) Overview	12
4	Live Map Dashboard – Bengaluru Overview	16
5	Area Filter System with Color-Coded Boundaries	16
6	Entity Drill-Down – Restaurant Detail Modal	17
7	Entity Drill-Down – Delivery Agent Tracker	17
8	Analytics Dashboard Page	18

CHAPTER 1

INTRODUCTION

1.1 Background

Large-scale food delivery platforms like Zomato generate millions of operational data points daily across restaurants, delivery agents, and customers. The operational landscape suffers from critical data fragmentation — restaurant data and delivery agent data exist in separate, siloed datasets with no shared schema or unified view, making cross-entity analysis nearly impossible. The absence of a centralized analytics interface leaves operations teams without the visibility required to make data-driven decisions.

This project addresses these challenges by building a Real-Time Food Operations Analytics Dashboard using a unified PostgreSQL relational database, a secure RESTful backend API, and an interactive Next.js geospatial frontend for the city of Bengaluru, India.

1.2 Purpose

The purpose of this project is to design and develop a comprehensive, real-time operational command center for a Zomato-modeled food delivery platform. The system serves as a practical application of DBMS concepts taught in the course — including normalization, ER modeling, SQL queries, indexing, and transaction management — in a production-grade, full-stack environment.

1.3 Objectives

- Design a normalized (3NF) relational PostgreSQL schema for Users, Restaurants, DeliveryAgents, and Orders with proper primary and foreign key constraints.
- Integrate and clean existing restaurant and delivery agent datasets into the unified database using data insertion scripts.
- Develop a secure RESTful backend API with JWT authentication, health-check endpoint, and modular route architecture.
- Build an interactive Next.js frontend dashboard with a MapLibre GL map showing restaurants, delivery agents, order destinations, and area-level filters.
- Implement animated, traffic-aware agent movement along real OSRM road routes at 60fps.

- Enable entity-level drill-down — clicking a restaurant or agent pin opens a detail popup with orders and metrics.
- Support area-based filtering (BTM Layout, Bannerghatta Road, Yelahanka, Brookefield, Hebbal, All Bengaluru) with map fly-to and color-coded boundary overlays.
- Display a live statistics panel showing total restaurants, agents, orders, and agent status breakdown (Available / On Delivery / Offline).

1.4 Existing System and Disadvantages

Existing food delivery operations platforms typically present the following critical gaps:

- Restaurant data and delivery agent data exist in separate, siloed datasets with no shared schema or unified view, making cross-entity analysis impossible.
- There is no geospatial visibility into where active restaurants, agents, and customer delivery locations are distributed across a city in real time.
- Operations teams cannot drill down into a specific restaurant's recent orders or track a delivery agent's active assignment without manual cross-referencing across multiple data sources.
- There is no centralized analytics interface to filter, monitor, or visualize the live state of the delivery network — making data-driven operational decisions impossible. This results in degraded service quality, delayed incident response, and missed optimization opportunities.

1.5 Proposed System and Advantages

The proposed system addresses each problem above through four integrated layers:

- Unified Database: A normalized PostgreSQL database (3NF) consolidates all four entities — Users, Restaurants, DeliveryAgents, and Orders — with well-defined foreign key relationships and enum-typed status fields.
- Type-Safe Backend API: A Node.js + Express server with Drizzle ORM exposes endpoints for map data, restaurant drill-down, agent drill-down, and order analytics — all protected by JWT authentication middleware.
- Interactive Geospatial Dashboard: A Next.js + MapLibre GL frontend renders animated delivery agent positions along real road routes (fetched from OSRM), restaurant pins, and customer delivery location markers — all on a live Bengaluru city map.

- **Real-Time Simulation:** Delivery agents tagged as 'On Delivery' animate along actual road geometries at traffic- and vehicle-adjusted speeds, simulating a live operations view with a 30-second data refresh cycle.

CHAPTER 2

LITERATURE REVIEW

The development of real-time operational dashboards for food delivery platforms draws upon several areas of prior research and industry practice.

2.1 Geospatial Data Management in Food Delivery

Studies on urban food delivery operations highlight the importance of geospatial data management for last-mile logistics. Systems like Google Maps Platform and HERE Maps have demonstrated that routing real delivery agents through live road networks — accounting for traffic density and vehicle type — significantly reduces average delivery times. Our use of the OSRM (Open Source Routing Machine) open-source routing engine for animated agent movement aligns with this body of work.

2.2 Database Normalization for Operational Analytics

Ramakrishnan and Gehrke (Database Management Systems, 3rd Edition) establish that 3NF normalization is the industry standard for transactional databases requiring high referential integrity. Multi-entity food delivery schemas storing Users, Restaurants, Delivery Agents, and Orders in normalized tables with proper foreign keys eliminate redundancy and support efficient aggregation queries — a design choice validated by platforms such as Swiggy and Deliveroo in their published engineering blogs.

2.3 RESTful API Design with JWT Authentication

The adoption of stateless JWT (JSON Web Tokens) authentication for REST APIs is well-established in the microservices literature. RFC 7519 defines the JWT standard, and its use in Express.js backend applications is widely documented. Our implementation of JWT middleware guarding all analytical endpoints follows best practices as described in the Express.js security documentation.

2.4 Real-Time Web Dashboards

Next.js 16 and React 19 represent the current state of the art for server-side rendered, highly interactive web dashboards. Combined with Zustand for global state management, the architecture supports a 30-second polling refresh cycle that keeps operational data current without the complexity of WebSocket infrastructure. Similar patterns are documented in the Vercel engineering blog and the React 19 release notes.

2.5 MapLibre GL for Open-Source Mapping

MapLibre GL JS is the open-source fork of Mapbox GL JS, providing GPU-accelerated vector tile rendering in the browser. Its use for food delivery operational maps has been documented in logistics startups transitioning away from proprietary mapping APIs. The ability to overlay custom GeoJSON markers, animate agent positions, and apply area-specific color themes makes it ideal for the operational command center use case presented in this project.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Specification
Processor	Intel Core i5 (8th Gen) or above
RAM	Minimum 8 GB (16 GB recommended)
Storage	Minimum 128 GB SSD
Display	1366×768 resolution or higher
Network	Internet access for OSRM routing API and npm packages

3.2 Software Requirements

Layer	Technology	Version / Notes
Database	PostgreSQL	v15 or later with native enum support
ORM	Drizzle ORM (TypeScript)	Type-safe schema definition and query building
Backend	Node.js + Express.js	Node v18.x or later, Express v4.x
Frontend	Next.js + React	Next.js 16, React 19
Mapping	MapLibre GL JS	v4.x — open-source vector map rendering
Routing	OSRM	Public API — real road geometries
UI Components	shadcn/ui + Tailwind CSS	v4 — component library and styling
Charts	Recharts	Analytics section charts
State	Zustand	Global client-side state management
Auth	JWT (jsonwebtoken)	Stateless token-based API authentication
Data Gen.	Python + Faker	Synthetic user and order data generation
Data Proc.	Python + Pandas	Data cleaning and standardization
Dev Tools	VS Code, Postman, Git	Code editing, API testing, version control

3.3 Functional Requirements

- FR-01: The system shall display a live geospatial map of Bengaluru with restaurant pins, delivery agent pins, and customer delivery location pins.
- FR-02: The system shall animate delivery agents tagged 'On Delivery' along real OSRM road routes at traffic- and vehicle-adjusted speeds.
- FR-03: The system shall support area-based filtering for BTM Layout, Bannerghatta Road, Yelahanka, Brookefield, Hebbal, and All Bengaluru.
- FR-04: The system shall provide entity drill-down — clicking a pin shall open a detail modal with relevant metrics.
- FR-05: The system shall display a live statistics panel with counts of total restaurants, agents, orders, and agent status breakdown.
- FR-06: All API endpoints shall be protected by JWT authentication middleware with a Zod-validated login form.

3.4 Non-Functional Requirements

- NFR-01 Performance: Agent animation shall render at 60fps with 50ms tick debounce; data shall refresh every 30 seconds.
- NFR-02 Scalability: PostgreSQL indexes on city, status, and coordinate columns shall support performant map queries at scale.
- NFR-03 Security: All analytical routes shall be JWT-protected; passwords shall be hashed and never stored in plaintext.
- NFR-04 Maintainability: The backend shall follow a modular route architecture; the frontend shall use reusable React components and Zustand global state.

CHAPTER 4

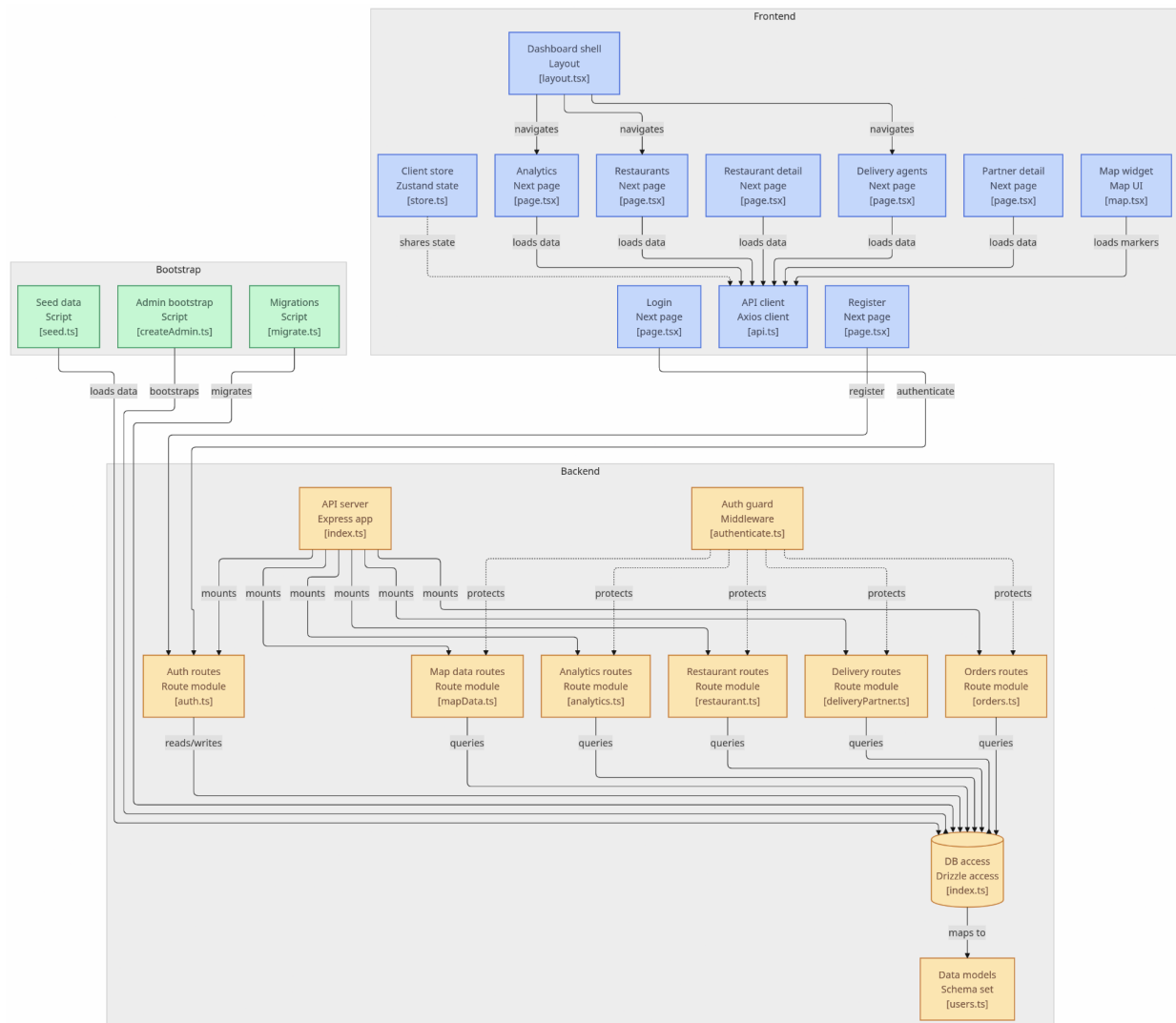
SYSTEM ARCHITECTURE

4.1 System Design

The system follows a three-tier client-server architecture:

- **Presentation Tier:** Next.js 16 / React 19 frontend with MapLibre GL map, area filter pill bar, live statistics panel, and entity detail popups. Communicates with the backend via Axios (api.ts client).
- **Application Tier:** Node.js + Express REST API with Drizzle ORM. Exposes GET /api/map-data, GET /api/restaurant/:id, GET /api/delivery-partner/:id, POST /api/auth/login, and GET /health. All data routes are JWT-protected via Auth Guard Middleware (authenticate.ts).
- **Data Tier:** PostgreSQL database storing all normalized operational data with Drizzle-managed schema migrations.
- **Routing Layer (External):** OSRM public routing API fetches real road geometries for 'On Delivery' agents, with a module-level route cache to minimize repeat API calls.

Bootstrap scripts Seed (seed.ts), Admin Bootstrap (createAdmin.ts), and Migration (migrate.ts) handle initial data loading and schema setup. The system architecture component relationship diagram (Fig. 1) illustrates the complete component relationships across the Frontend, Backend, and Database tiers.



4.2 Database Design

4.2.1 Normalization

The database is designed in Third Normal Form (3NF) to eliminate redundancy, ensure referential integrity, and optimize query performance. Each entity resides in its own table with a surrogate serial primary key and a domain-specific string ID.

4.2.2 Entity Schema

Entity	Key Attributes
Users	user_id (PK), first_name, last_name, email, phone, default_latitude, default_longitude, city, location_area, created_at
Restaurants	restaurant_id (PK), name, address, location_neighborhood, latitude, longitude, rate, votes, cuisines, rest_type, online_order, listed_in_city

Entity	Key Attributes
DeliveryAgents	delivery_person_id (PK), first_name, last_name, phone, age, ratings, vehicle_type (enum), status (enum), current_latitude, current_longitude, total_deliveries, city
Orders	order_id (PK), restaurant_id (FK), delivery_person_id (FK), user_id (FK), restaurant_latitude, restaurant_longitude, delivery_location_latitude, delivery_location_longitude, order_date, time_taken_min, weather_conditions (enum), road_traffic_density (enum), type_of_order (enum), type_of_vehicle, festival

4.2.3 Enums Used

Enum Name	Values
vehicle_type	motorcycle, scooter, electric_scooter, bicycle
agent_status	Available, On Delivery, Offline
weather_conditions	Sunny, Cloudy, Rainy, Foggy, Windy, Stormy
road_traffic_density	Low, Medium, High, Jam
order_type	Snack, Meal, Drinks, Buffet

CHAPTER 5

IMPLEMENTATION

5.1 Project Roadmap and Implementation Steps

Phase 1: Data Preparation

Dataset analysis was performed on publicly available Zomato restaurant data for Bengaluru and a delivery agent dataset. Python scripts using the Pandas library were written to clean and standardize the datasets — handling missing values, normalizing address formats, and filtering records to the six target Bengaluru areas. The Python Faker library was used to generate synthetic User and Order records with realistic geospatial coordinates, time-taken values, and categorical enum fields.

Phase 2: Database Setup

The 3NF schema was designed using Drizzle ORM's TypeScript schema definition (users.ts). The schema includes all four entities with proper primary keys, foreign key references, and native PostgreSQL enum types. Migration scripts (migrate.ts) were run to apply the schema to a local PostgreSQL v15 instance. Bulk CSV data was inserted using the seeding script (seed.ts), which reads cleaned CSV files and performs batched inserts with transaction wrapping for consistency.

Phase 3: Backend API Development

An Express.js REST API server (index.ts) was structured with modular route files — auth.ts, mapData.ts, restaurant.ts, deliveryPartners.ts, analytics.ts, and orders.ts. JWT authentication middleware (authenticate.ts) was applied as a guard on all data routes. The map-data endpoint joins Restaurants, DeliveryAgents, and the latest Orders in a single optimized Drizzle query. The restaurant and delivery-partner detail endpoints fetch entity data along with associated order histories.

Phase 4: Frontend Dashboard Development

The Next.js 16 app was structured with a dashboard shell layout (layout.tsx) and dedicated pages for /restaurants, /delivery-agents, /analytics, /restaurant/:id, and /delivery-partner/:id. The MapLibre GL map (map.tsx) was configured with a Bengaluru center coordinate and custom GeoJSON layers for restaurant pins (rose), agent pins (green/amber/grey by status), and delivery destination pins (sky blue). The area filter pill bar triggers map fly-to animations and API-level area filtering. Agent animation uses a 60fps requestAnimationFrame loop with interpolated positions along OSRM road geometry, debounced to 50ms ticks. Zustand (store.ts) manages global state including active filters, selected entities, and fetched map data.

5.2 DBMS Concepts Applied

DBMS Concept	Application in Project
Normalization (3NF)	Schema design across all four tables eliminates redundancy and update anomalies
ER Modeling	Entity-Relationship diagram mapping Users, Restaurants, DeliveryAgents, Orders
Primary & Foreign Keys	Orders table references all three other entities via FK with referential integrity
Enum Types	PostgreSQL native enums for vehicle_type, agent_status, weather_conditions, road_traffic_density, order_type
SQL JOIN Queries	GET /api/restaurant/:id fetches restaurant + related orders; agent endpoint fetches agent + assignments
Aggregations	Stats endpoint aggregates total counts and status breakdowns across tables
Indexing	Indexes on city, status, and coordinate columns for performant map queries
ORM Integration	Drizzle ORM provides type-safe, programmatic schema definition and query building in TypeScript
Transactions	Atomic operations for order placement ensuring consistency across all four tables
JWT Authentication	Token-based stateless auth protecting all analytical API routes

5.3 Team Member Contributions

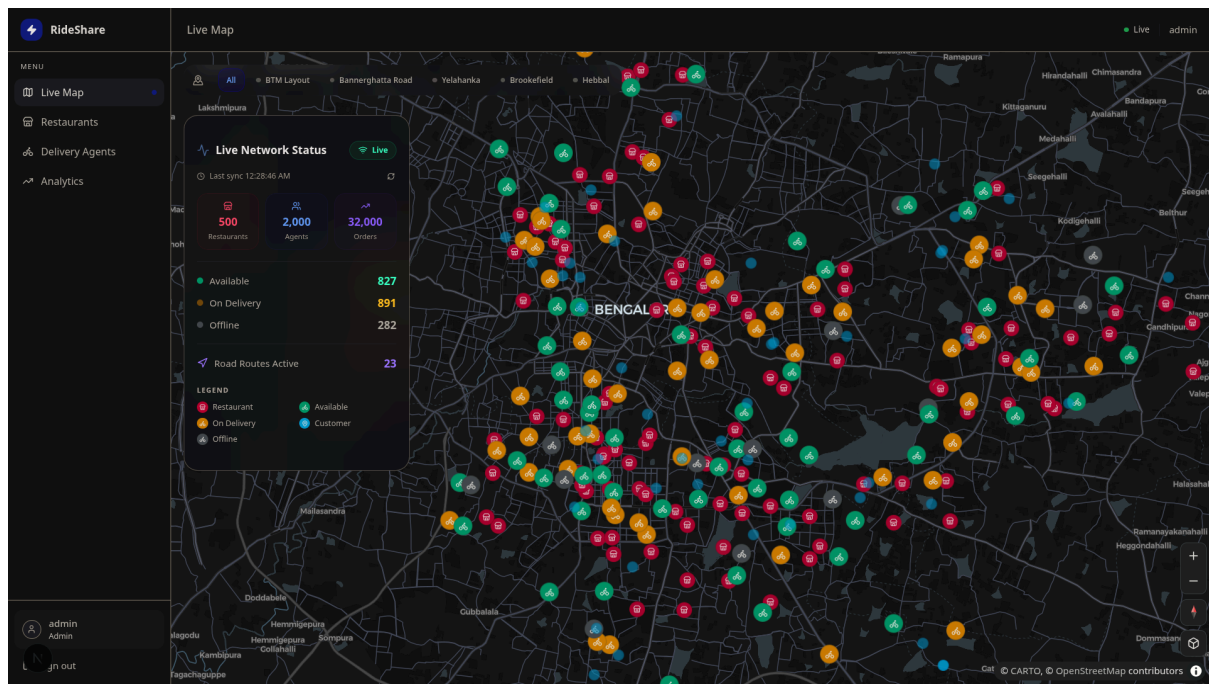
Member	Responsibility
Pavan C / Prajjwal Sahu	Data cleaning, Python scripts, Faker-based data generation for Users and Orders
Prajjwal Sahu	Next.js frontend, MapLibre GL map integration, animated agent movement, area filters, UI/UX
Niteesh Srivathsa / Prajjwal Sahu	PostgreSQL schema design (3NF), Drizzle ORM setup, database seeding and migrations
Prajwal Mantur / Prajjwal Sahu	Node.js/Express backend API — auth, map-data, restaurant, delivery-partner routes
Prajwal Babu	Testing, ER diagram, project report, API documentation, presentation

CHAPTER 6

RESULTS AND SCREENSHOTS

The following screenshots illustrate the key features of the Real-Time Food Operations Analytics Dashboard deployed on a local development environment with Bengaluru seeded data.

Screenshot 1: Live Map Dashboard – Bengaluru Overview



The live map dashboard renders all entities simultaneously. Restaurant pins appear in red, delivery agents in green (Available), amber (On Delivery), and grey (Offline), and customer delivery destinations in sky blue. The top-right statistics panel displays real-time counts and last-sync timestamp.

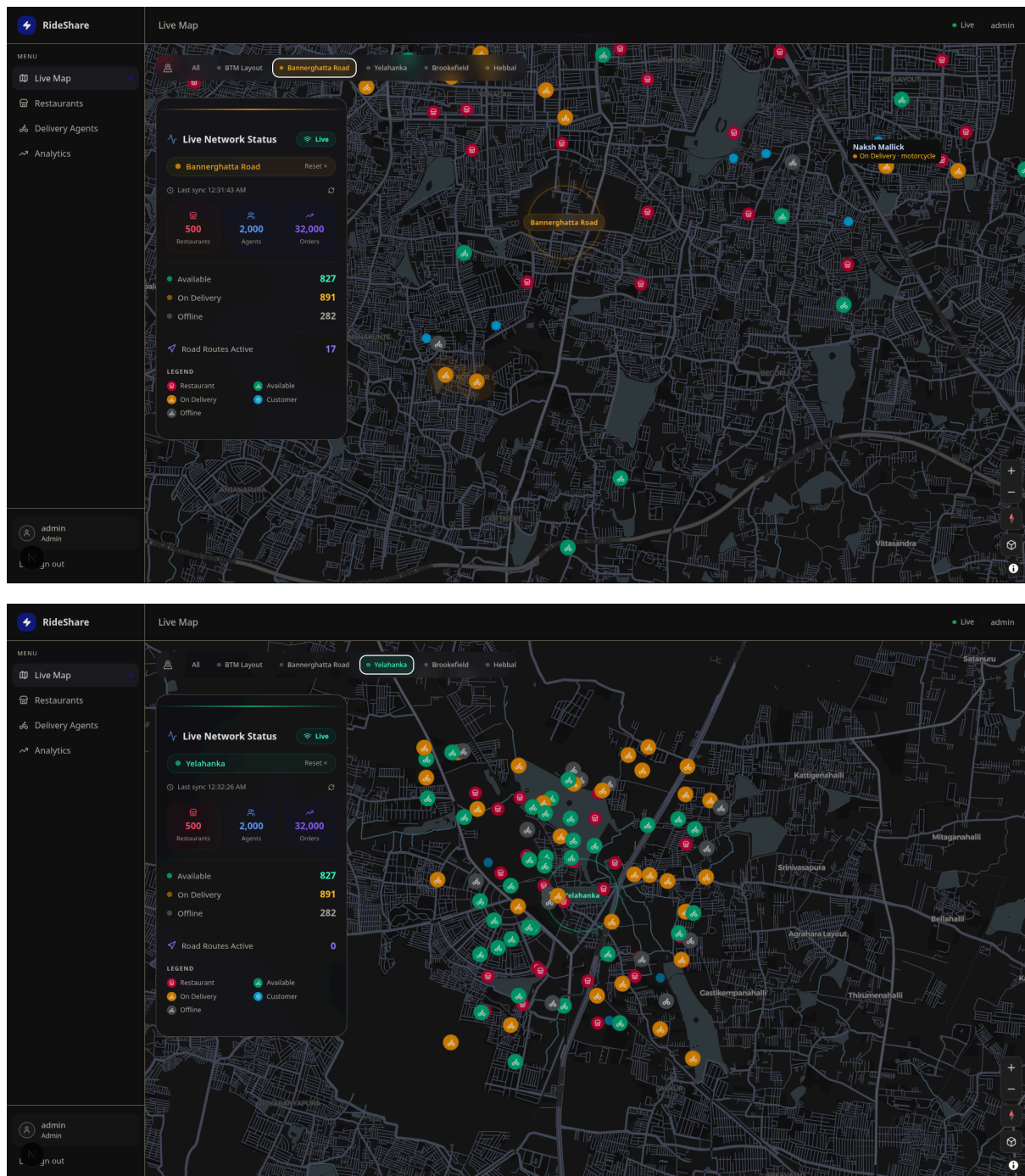
Screenshot 2: Area Filter System

Selecting an area filter (e.g., BTM Layout) triggers a smooth map fly-to animation, renders pulsing boundary ring markers in the area's theme color, and filters the API data to show only entities in that zone. The pill bar displays all six areas with their

distinct

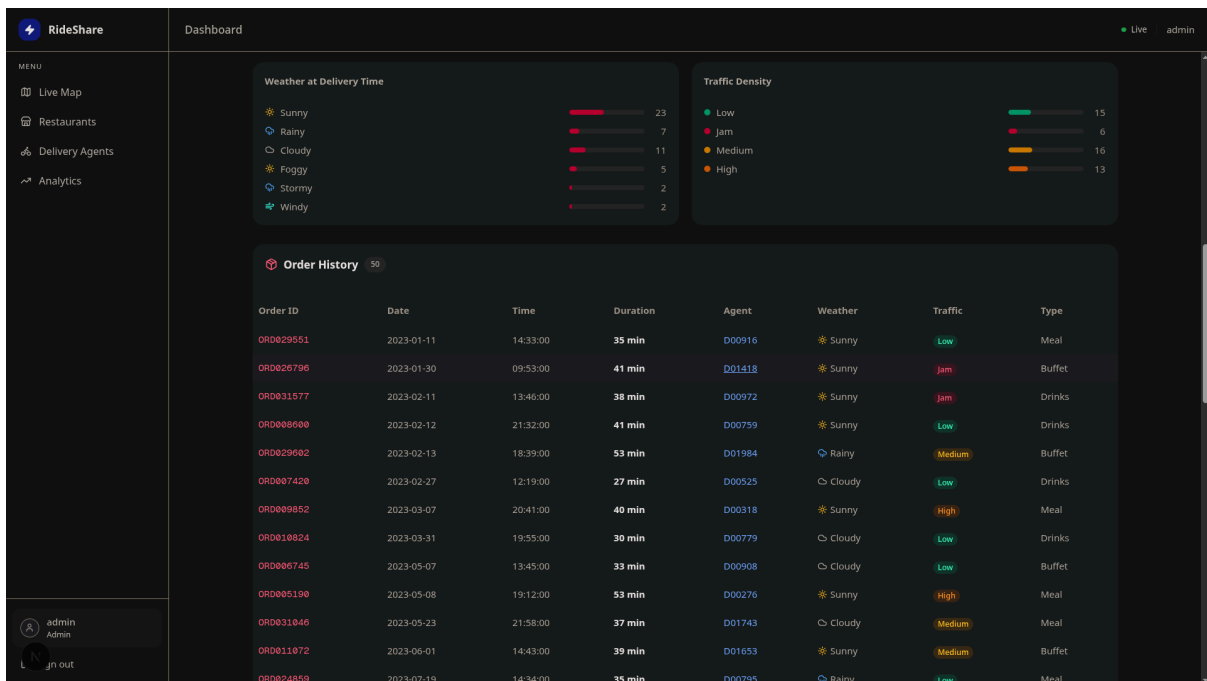
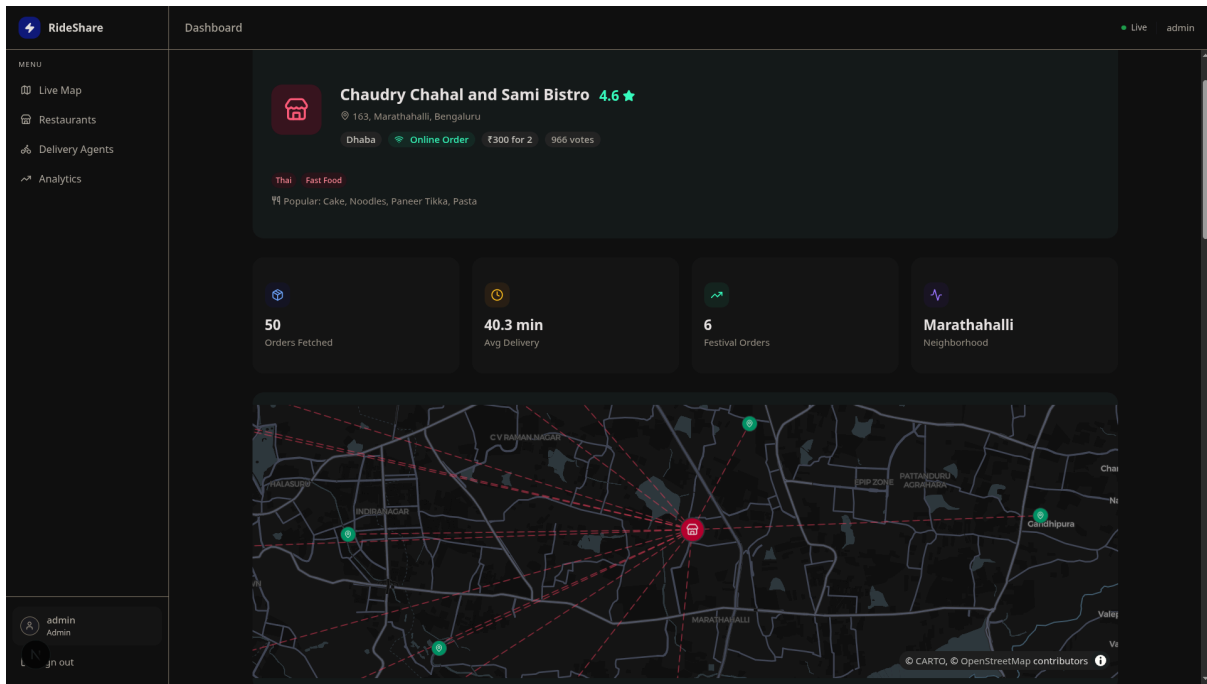
color

themes.



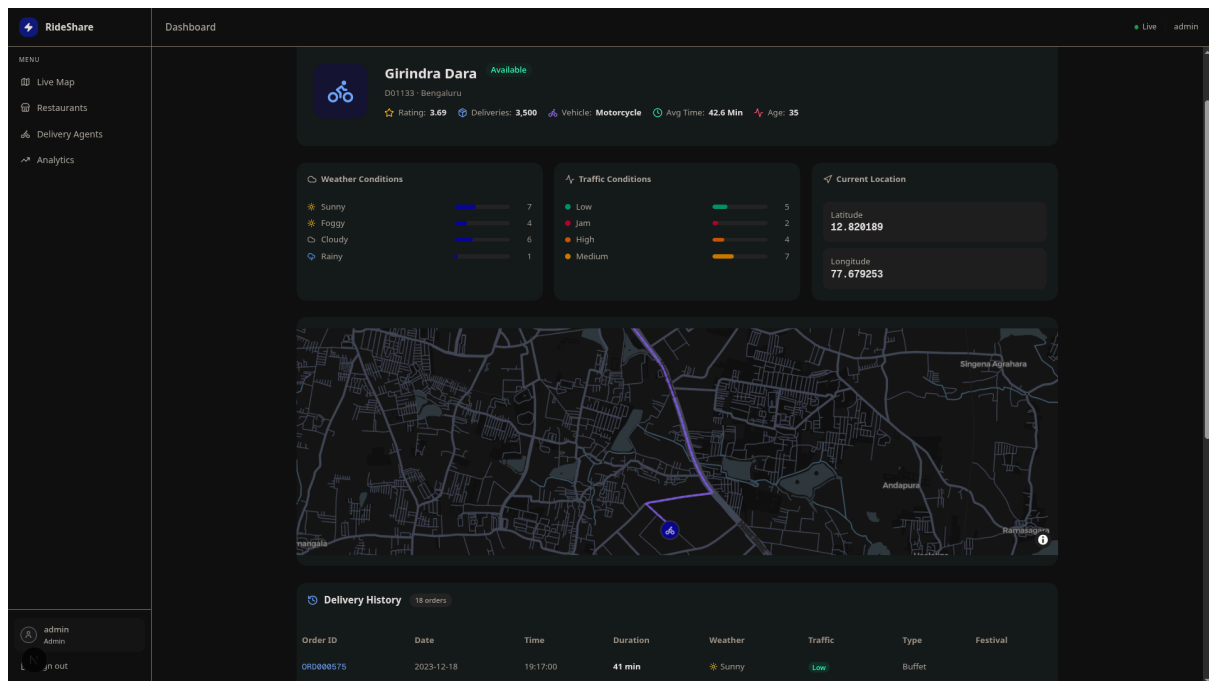
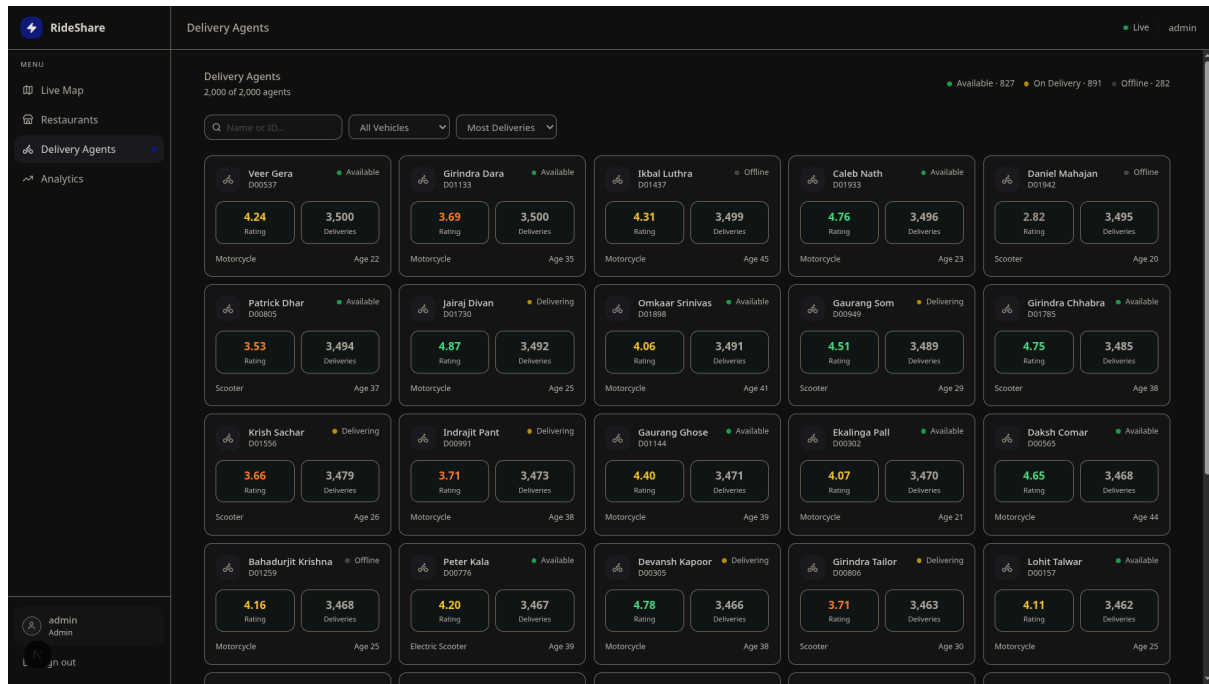
Screenshot 3: Entity Drill-Down – Restaurant and Delivery Agent Modals

Clicking a restaurant pin opens a modal displaying the restaurant name, area, rating, cuisine types, and a list of recent orders with statuses. Clicking an agent pin shows the agent's status, vehicle type, rating, and the active order ID. Both modals link to dedicated detail pages at </restaurant/:id> and </delivery-partner/:id>.



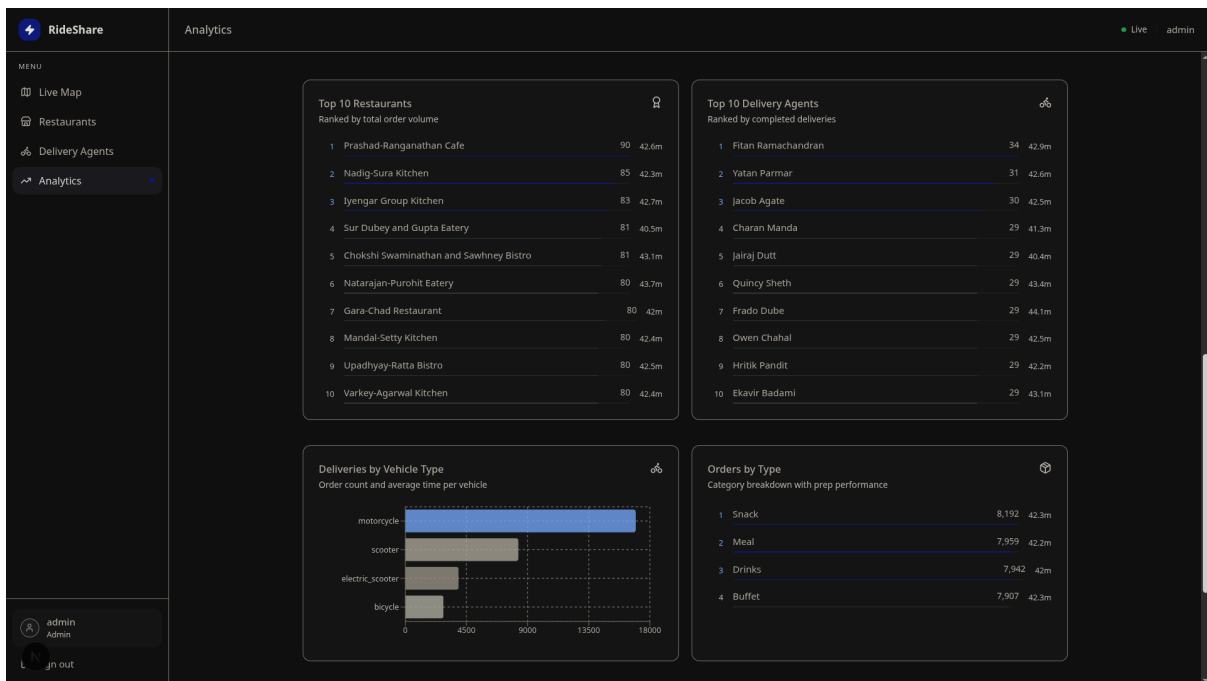
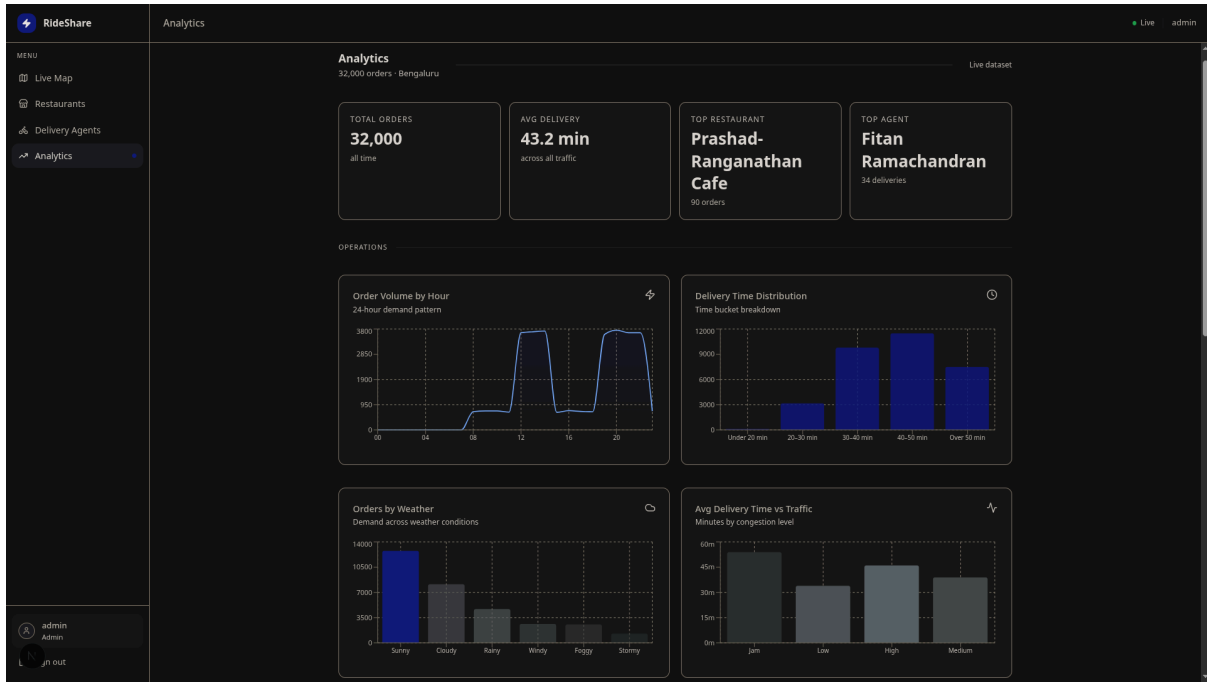
Screenshot 4: Delivery Riders Page

The dedicated Delivery Riders page (/riders) presents a comprehensive overview of all active delivery agents, including rider profiles, real-time status, vehicle type assignments, and individual performance metrics. Data is fetched from the riders REST endpoint with Drizzle-aggregated PostgreSQL queries.



Screenshot 5: Analytics Dashboard Page

The dedicated analytics page (/analytics) presents Recharts-powered visualizations including order volume over time, road traffic density distribution across orders, vehicle type breakdown across delivery agents, and weather conditions affecting delivery time. Data is fetched from the analytics REST endpoint with Drizzle-aggregated PostgreSQL queries.



CONCLUSION AND FUTURE ENHANCEMENT

Conclusion

This project successfully designed and implemented a Real-Time Food Operations Analytics Dashboard for a Zomato-modeled food delivery platform as a comprehensive application of DBMS course concepts. A normalized PostgreSQL database in Third Normal Form (3NF) provides a unified operational data layer for Users, Restaurants, DeliveryAgents, and Orders with proper referential integrity. A secure Node.js + Express RESTful API with JWT authentication and Drizzle ORM exposes this data to an interactive Next.js + MapLibre GL frontend that visualizes live agent positions, restaurant locations, and delivery destinations across Bengaluru. The system demonstrates practical application of normalization, ER modeling, SQL JOINS, aggregations, indexing, enum types, and transaction management in a production-grade full-stack environment.

Future Enhancement

- **WebSocket Integration:** Replace the 30-second polling refresh cycle with WebSocket-based push updates for true real-time data delivery.
- **Predictive Analytics:** Integrate machine learning models to predict delivery time based on weather conditions, traffic density, vehicle type, and historical order data stored in the database.
- **Multi-City Support:** Extend the area filter system to support additional Indian cities beyond Bengaluru, with city-level database partitioning.
- **Mobile Application:** Develop a React Native mobile companion app for delivery agents to update their status and receive order assignments.
- **Advanced Query Optimization:** Implement PostgreSQL materialized views and query caching for the analytics aggregation endpoints to reduce database load at scale.

REFERENCES

- [1] Ramakrishnan, R. & Gehrke, J. (2002). Database Management Systems, 3rd Edition. McGraw-Hill.
- [2] PostgreSQL Global Development Group. (2024). PostgreSQL 15 Official Documentation. <https://www.postgresql.org/docs/>
- [3] Drizzle ORM Team. (2024). Drizzle ORM Documentation — Type-Safe TypeScript ORM. <https://orm.drizzle.team/>
- [4] MapLibre Contributors. (2024). MapLibre GL JS Documentation. <https://maplibre.org/maplibre-gl-js/docs/>
- [5] OpenJS Foundation. (2024). Express.js Official Guide. <https://expressjs.com/>
- [6] Vercel, Inc. (2024). Next.js 16 Documentation. <https://nextjs.org/docs>
- [7] Project OSRM. (2024). OSRM Routing API Documentation. <https://project-osrm.org/>
- [8] Python Software Foundation. (2024). Python Faker Library Documentation. <https://faker.readthedocs.io/>
- [9] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force.
- [10] Wieruch, R. (2023). The Road to React — Full-Stack Applications with Next.js. Leanpub.
- [11] Silberschatz, A., Korth, H.F., & Sudarshan, S. (2019). Database System Concepts, 7th Edition. McGraw-Hill.
- [12] Recharts Team. (2024). Recharts — Composable Charting Library for React. <https://recharts.org/>
- [13] McKinney, W. (2022). Python for Data Analysis: Data Wrangling with Pandas, NumPy & Jupyter, 3rd Edition. O'Reilly.
- [14] Zomato Engineering Blog. (2023). Scaling Food Delivery Operations with Real-Time Data Pipelines. Zomato Tech Blog.
- [15] Shadcn. (2024). shadcn/ui — Beautifully Designed Components. <https://ui.shadcn.com/>